Introduzione

Architettura ARM (Advanced RISC Machine)

Un processore con **architettura ARM** fa parte di una famiglia di processori di tipo **RISC** (**R**educed **I**nstruction **S**et **C**omputer) generalmente con registri a **32-bit**. L'architettura di tipo RISC garantisce ottime prestazioni con un basso consumo energetico, per questo i processori ARM trovano un largo uso nel mercato dei dispositivi mobili dove il risparmio energetico è di fondamentale importanza. I processori ARM si trovano nei più comuni dispositivi come cellulari, tablet, videogiochi portatili, raspberry, ecc…

Registri

I processori ARM hanno in totale 37 registri a 32-bit, organizzati come segue.

16 registri, denominati da R0 a R15, distinti in base al loro utilizzo:

- R0-R12 sono i registri di uso generale.
- R13 viene usualmente usato come **Stack Pointer SP**, ma non è obbligatorio. Lo *stack* è una struttura dati molto importante la quale alloca memoria in maniera dinamica.
- R14 ha la funzione di **subroutine Link Register LR**, al cui interno ci viene salvato l'indirizzo di ritorno ovvero l'indirizzo del PC al momento dell'esecuzione dell'istruzione BL (**Branch and Link**). Quindi quando viene chiamata una routine il registro LR viene caricato con l'indirizzo dell'ultima istruzione al momento della chiamata. Quando la routine terminerà il PC verrà caricato con l'indirizzo all'interno di LR per **ritornare** alla procedura *chiamante*.
- R15 ha la funzione di **Program Counter PC**, contiene l'indirizzo della prossima istruzione da eseguire.

R0	0x00000000
R1	0x00000000
R2	0x00000000
R3	0x00000000
R4	0x00000000
R5	0x00000000
…	…
SP	0x00000000
LR	0x00000000
PC	0x00000000

Un **registro di stato CPSR** (**C**urrent **P**rogram **S**tatus **R**egister) molto utilizzato per definire uno stato in seguito a un confronto. Nello specifico per i confronti il registro CPSR dedica gli ultimi 4 bit più significati (detti *flag di stato*) per esprimere altrettante condizioni:

1. **N**egative
2. **Z**ero
3. **C**arry
4. o**V**erflow

In fine ci sono 20 registri "duplicati" (**banked**) dei registri precedenti.

Tipi di dati

I *tipi di dati* sulle quali le istruzioni operano sono:

- **Byte** 1-byte, 8-bit
- **Half word** 2-byte, 16-bit
- **Full word o word** 4-byte, 32-bit

Il linguaggio assemblativo può esprimere valori *numerici, booleani, stringhe* e *singoli caratteri*. Questi formati posso esprimere:

- Numeri decimali, per esempio 123.
- Numeri esadecimali, per esempio 0x7B.
- Numeri binari, per esempio 0b204.
- Numeri in qualsiasi base comprese tra 2 e 9, per esempio 5_204 è il numero 54 espresso in base 5.
- Numeri in virgola mobile, per esempio 123.4.
- Valori booleani {TRUE} o {FALSE}.
- Singoli caratteri racchiusi in singoli apici, per esempio 'w'.
- Stringhe espresse come insiemi di caratteri racchiusi in doppi apici, per esempio "This is a string"

Set di istruzioni

Le istruzioni dell'architettura ARM possono essere suddivise in tre insiemi:

i. **Istruzioni aritmetico-logiche** implementano le operazioni matematiche. Queste possono essere di tipo *aritmetiche* (somma, differenza, prodotto), *logiche* (operazioni booleane) o *relazionali* (comparazioni tra due valori).
ii. **Istruzioni di Branch** cambiano il *controllo del flusso* delle istruzioni, modificando il contenuto del Program Counter (R15). Le istruzioni di branch sono necessarie per l'implementazione di *istruzioni di condizione* (if then else), *cicli* e *chiamate di funzioni*.
iii. **Istruzioni di Load/Store** muovono dati da (*load*) e verso (*store*) la memoria principale.

Per comodità di decodifica tutte le istruzioni ARM sono su 32 bit e la manipolazione dei dati avviene *solamente* nei registri.

Nonostante una architettura di tipo RISC le istruzioni supportano diverse modalità di indirizzamento quali indirizzamento *immediato (valore numerico* espresso in maniera espicita*), a registro (contenuto del registro), diretto (indirizzo di memoria), a registro indiretto (puntatore)* e *indicizzato esteso (indirizzo + offset)*.

Un'altra caratteristica importante delle istruzioni ARM è l'**esecuzione condizionale**. I primi 4 bit più significativi delle istruzioni contengono una *condizione*. L'istruzione verrà quindi eseguita se e solo se corrisponde alla condizione presente nei flag di stato del CPSR.

Istruzioni aritmetico-logiche

Le istruzioni di tipo aritmetico-logiche hanno una *sintassi* del tipo

$$opcode\ \{S\}\{condition\}\ \ dest, op1, op2, \{shif\ operand, \#val\}$$

Per $opcode$ si intende il nome dell'istruzione che verrà eseguita, mentre il campo $\{condition\}$ è opzionale e definisce la condizione richiesta per l'esecuzione dell'istruzione. Se la condizione è omessa allora l'istruzione verrà eseguita sempre.

Il campo $dest$ indica il *registro destinatario* nel quale verrà caricato il risultato dell'operazione.

I campi $op1$ e $op2$ indicano gli *operandi* delle operazioni.

Un altro campo **molto** importante delle istruzioni aritmetico logiche è $\{shif\ operand, \#val\}$ il quale se specificato esegue una operazione di *shift logico* sul secondo operando prima di eseguire l'operazione aritmetico-logica indicata dall'$opcode$.

In fine il campo $\{S\}$ è un suffisso opzionale che, se specificato, carica nei *flag di stato* lo stato del risultato dell'operazione.

Per comodità faremo riferimento ai registri *destinatario, op1* e *op2* rispettivamente con R_d, R_1 e R_2.

Istruzioni di somma ADD, ADC

Esistono due istruzioni che implementano l'operazione aritmetica di somma, ADD *addizione* e ADC *addizione con riporto* (*add with carry*).

Descrizione	**Opcode**	**Sintassi**	**Semantica**
Addizione	ADD	ADD R_d, R_1, R_2/#imm	$R_d = R_1 + R_2$/#imm
Addizione con riporto	ADC	ADC R_d, R_1, R_2/#imm	$R_d = R_1 + R_2$/#imm + carry

Le istruzioni di somma caricheranno in R_d il risultato dell'addizione tra il primo e il secondo operando. Notiamo che il primo operando è un *registro* perciò indicherà il contenuto del registro indicato (*indirizzamento a registro*). Invece il secondo operando può essere indirizzato sia tramite un *registro* (*indirizzamento a registro*) sia tramite un valore numerico (*indirizzamento immediato*). Per esprimere un valore numerico in maniera immediata bisogna inserire il prefisso '#' al valore che si desidera esprimere.

Esempio:

ADD R0, R1, R2

Carico in R0 la somma tra il contenuto del registro R1 e del registro R2 (*indirizzamento a registro*).

ADD R0, R1, #16

Carico in R0 la somma tra il contenuto del registro R1 e il numero decimale 16 (*indirizzamento immediato*).

ADD R0, R1, #0xF

Carico in R0 la somma tra il contenuto del registro R1 e il numero esadecimale F (*indirizzamento immediato*).

ADDLT R0, R1, R2

Carico in R0 la somma tra il contenuto del registro R1 e del registro R2 solamente se la condizione indicata con il suffisso **LT** è verificata.

ADD R0, R1, #16, LSR #2

Carico in R0 la somma tra il contenuto del registro R1 e il numero decimale 16 "shiftato" a destra di 2 bit. Il valore decimale del secondo operando sarà quindi 4

ADDS R0, R1, R2

Carico in R0 la somma tra il contenuto del registro R1 e del registro R2, inoltre carico lo stato del risultato di flag del CPSR. Se per esempio il risultato va in overflow avremo che il flag C sarà aggiornato a 1.

Istruzioni di differenza SUB, SBC, RSB, RSC

Analogamente per la somma esistono due istruzioni che implementano l'operazione aritmetica della differenza, SUB *sottrazione* e SBC *sottrazione con riporto* (*subtract with carry*). Esistono inoltre altre due istruzioni di sottrazione importanti, RSB *reverse subtract* e RSB *reverse subtract with carry*. Queste ultime due istruzioni eseguono una operazione di sottrazione (con o senza riporto) ma caricano nel registro destinatario il suo *inverso*.

Descrizione	Opcode	Sintassi	Semantica
Sottrazione	SUB	SUB R_d, R_1, R_2/#imm	$R_d = R_1 - R_2$/#imm
Sottrazione con riporto	SBC	SBC R_d, R_1, R_2/#imm	$R_d = R_1 - R_2$/#imm -1 + carry
Sottrazione e inverso	RSB	RSB R_d, R_1, R_2/#imm	$R_d = -(R_1 - R_2$/#imm)
Sottrazione e inverso con riporto	RSC	RSC R_d, R_1, R_2/#imm	$R_d = -(R_1 - R_2$/#imm) -1 + carry

Esempio:

SUB R0, R1, R2

Carico in R0 la differenza tra il contenuto del registro R1 e del registro R2 (*indirizzamento a registro*).

SUB R0, R1, #16

Carico in R0 la differenza tra il contenuto del registro R1 e il numero decimale 16 (*indirizzamento immediato*).

SUB R0, R1, #0xF

Carico in R0 la differenza tra il contenuto del registro R1 e il numero esadecimale F (*indirizzamento immediato*).

SUBLT R0, R1, R2

Carico in R0 la differenza tra il contenuto del registro R1 e del registro R2 solamente se la condizione indicata con il suffisso **LT** è verificata.

RSB R0, R1, R2

Carico in R0 l'inverso della differenza tra il contenuto del registro R1 e del registro R2.

SUB R0, R1, R2, LSL #3

Carico in R0 la differenza tra il contenuto del registro R1 e del registro R2 però shiftato a sinistra di 3 bit.

SUBS R0, R1, R2

Carico in R0 la differenza tra il contenuto del registro R1 e del registro R2, inoltre carico lo stato del risultato di flag del CPSR. Se per esempio R1 e R2 hanno lo stesso valore allora la differenza sarà uguale a 0, quindi il *flag* Z verrà posto a 1. Invece se la differenza ha come risultato un valore negativo il *flag* N verrà posto a 1.

Istruzione di prodotto MUL

L'unica istruzione che implementa la moltiplicazione è MUL descritta come segue

Descrizione	Opcode	Sintassi	Semantica
Moltiplicazione	MUL	MUL R_d, R_1, R_2/#imm	$R_d = R_1 \times R_2$/#imm

Anche per l'istruzione MUL è possibile specificare una *condizione* e un eventuale *shift logico* del secondo operando.

Istruzione di inizializzazione registri MOV, MVN

Generalmente tutti i registri di uso generale sono inizializzati a 0. È possibile però aggiornare il loro valore, qualunque esso sia (anche 0), con un valore numerico qualsiasi. Per farlo si usa l'istruzione MOV *move* (letteralmente *muovi*) la quale carica nel registro un valore numerico. Una istruzione simile e MVN *move negated* la quale carica il negato del valore che si vuole caricare nel registro. Anche in questo caso è possibile specificare una *condizione* e un eventuale *shift logico* del valore che si desidera caricare.

Descrizione	Opcode	Sintassi	Semantica
Carica nel registro	MOV	MOV R_d, R_1/#imm	$R_d \leftarrow R_1$/#imm
Carica negato nel registro	MVN	MVN R_d, R_1/#imm	$R_d \leftarrow -(R_1$/#imm)

Esempio:

```
MOV   R0, #16
MOV   R1, R0
MVN   R2, R1, LSR #1
```

Registro	Contenuto
R0	16
R1	16
R2	-8

Operazioni logiche booleane AND, OR, XOR, BIC

Descrizione	Opcode	Sintassi	Semantica
AND logico	AND	AND R_d, R_1, R_2/#imm	$R_d = R_1 \wedge R_2$/#imm
OR logico	OR	ORR R_d, R_1, R_2/#imm	$R_d = R_1 \vee R_2$/#imm
OR Esclusivo	EOR	EOR R_d, R_1, R_2/#imm	$R_d = R_1 \oplus R_2$/#imm
AND con complemento del secondo operando	BIC	BIC R_d, R_1, R_2/#imm	$R_d = R_1 \wedge \sim(R_2$/#imm)

Le istruzioni che implementano le operazioni logiche booleane eseguono la rispettiva operazione descritta dall'opcode bit a bit per tutto il contenuto del registro. Anche queste istruzioni possono avere un campo per la *condition* e uno per l'operazione di *shift logico* del secondo operando.

Esempio:

```
MOV   R0, #0b0101
MOV   R1, #0b0011
AND   R2, R0, R1        // A ∧ B
ORR   R3, R0, R1        // A ∨ B
EOR   R4,  R0, R1       // A ⊕ B
BIC   R5,  R0, R1       // A ∧ ∽ B
```

Registro	Contenuto
R0	0101
R1	0011
R2	0001
R3	0111
R4	0110

R5	0100

Istruzioni di Shift e Rotazione

Un essenziale insieme di operazioni sono le operazioni di **Rotazione** e di **Scorrimento**, o **Shift**. Queste operazioni spostano una sequenza di bit di *n* posizioni verso destra o verso sinistra. Lo Shift scorre verso destra o sinistra i bit contenuti in una parola e i bit in uscita si ritengono perduti, mentre quelli in entrata sono tutti 0. Analogamente anche l'operazione di rotazione scorre verso una direzione i bit di una parola, con la differenza però che i bit in uscita equivalgono a quelli in entrata.

0000 0000 0011 1111	*Half word X*
0000 0011 1111 0000	*Shift logico sinistro di 4 bit*

0000 0000 0011 1111	*Half word X*
1110 0000 0000 0111	*Rotazione a destra di 4 bit*

Esiste anche una operazione di *scorrimento destro aritmetico*, dove i bit in entrata non sono sempre riempiti con 0 bensì equivalgono al valore dell'ultimo bit più a sinistra. Quindi se l'ultimo bit più a sinistra vale 0 allora tutti i bit in entrata varranno 0, se invece vale 1 allora tutti i bit in entrata varranno 1.

1010 0111 0011 1111	*Half word X*
1111 1010 0111 0011	*Shift aritmetico destro di 4 bit*

In fine esiste l'operazione di *rotazione destra con estensione*, la quale ruota di **un solo bit verso destra** la parola però nel bit in entrata (quello più a sinistra) viene caricato il valore del *flag carry* (cioè il bit di stato C del registro di stato CPSR). Invece il bit in uscita verrà caricato come nuovo valore del *flag carry*.

0000 0000 0000 1111	*Half word X*
0000 0000 0000 0111	*rotazione destra con estensione (con flag C = 0)*
1000 0000 0000 0111	*rotazione destra con estensione (con flag C = 1)*

Descrizione	**Opcode**	**Sintassi**
Shift logico sinistro	LSL	LSL R_d, R_1, R_2/#imm
Shift logico destro	LSR	LSR R_d, R_1, R_2/#imm
Shift destro aritmetico	ASR	ASR R_d, R_1, R_2/#imm
Rotazione destra	ROR	ROR R_d, R_1, R_2/#imm
Rotazione destra con estensione	RRX	RRX R_d, R_1

Istruzioni di confronto o relazionali

Le istruzioni di confronto sono **essenziali** per il controllo del flusso delle istruzioni, in quanto determina la *condizione* (*condition*) necessaria e sufficiente per l'esecuzione o meno di una istruzione. Infatti generalmente queste istruzioni eseguono un **confronto** tra due *parole* e aggiornano i *flag* NZCV del registro di stato.

Esistono due tipologie di istruzioni di confronto, quelle di **test** (**testing**) e quelle di **confronto** (**comparing**).

- ## Compare e Compare Negatet

L'istruzione CMP (*compare*) confronta i valori dei bit di due registri. In base al risultato l'istruzione aggiorna i *flag* NZCV del registro di stato ma non salva il risultato del confronto in nessun registro. In pratica CMP sottrae il valore del secondo operando R_2 dal primo operando R_1 (in pratica esegue R_1- R_2). CMP è quindi equivalente all'istruzione SUBS con la differenza che non ne salva il risultato.

Per esempio se R_1=5 e R_2=6 ed eseguiamo una *compare* tra i due registri avremo come condizione che R_1< R_2. Quindi i *flag* di stato esprimeranno una condizione che descrive che R_1< R_2. Se questa condizione è necessaria per l'esecuzione di una istruzione questa verrà eseguita.

Esempio:

```
MOV    R0, #5          //R0 = 5
MOV    R1, #6          //R1 = 6
CMP    R0, R1          //confronto R0 e R1 e scopro che R0 è minore stretto di R1
ADDLT  R2, R0, R1      // la condition LT farà eseguire ADD se e solo se i flag di stato descrivono
una condizione di minore stretto
SUBGT  R2, R0, R1      // la condition GT farà eseguire SUB se e solo se i flag di stato descrivono
una condizione di maggiore stretto
```

```
Codice C equivalente
1.      int R0 = 5;
2.      int R1 = 6;
3.      int R2;
4.
5.      if ( R0 < R1 ) {
6.          R2 = R0 + R1;
7.      } else {
8.          R2 = R0 - R1;
9.      }
```

Nell'esempio inizializzo i registri R0 e R1 con i rispettivi valori 5 e 6. Eseguo un confronto tra i due registri e aggiorno i valori dei *flag* NZCV; più nello specifico avremo che N = 1, Z = 0, C = 0 e V = 0. La *condition* LT farà eseguire l'istruzione ADD se e solo se R0 è minore stretto di R1. Dato che la condizione è verificata allora verrà eseguita l'istruzione ADD. Analogamente la *condition* GT farà eseguire l'istruzione SUB se e solo se R0 è maggiore stretto di R1. In questo caso SUB non verrà eseguita.

Analogamente l'istruzione CMN (*compare negated*) eseguirà una comparazione **negata** rispetto all'istruzione CMP. Infatti CMN è equivalente all'istruzione ADDS, con la differenza che il risultato non verrà salvato in alcun registro.

- ## Test bits set e Test Equals

Le operazioni di *testing* TST (*Test bit set*) e TEQ (*Test Equals*) confrontano se due valori sono uguali.

Nello specifico TST esegue una operazione di AND logico bit a bit dei valori tra R_1 e R_2, aggiornando il valore dei *flag* di condizione e senza salvare il risultato in nessun registro. L'istruzione TST è equivalente all'istruzione ANDS, con la differenza che non ne salva il risultato.

Analogamente l'istruzione TEQ esegue una operazione di OR Esclusivo bit a bit tra R_1 e R_2. Quindi TEQ equivale a EORS con la differenza che aggiorna solamente i *flag* di condizione. Questa operazione è utile per verificare se il *bit di segno* di un numero in complemento a due e uguale a un altro.

Descrizione	Opcode	Sintassi	Semantica
Confronto	CMP	CMP R_1, R_2	NZCV ← R_1 -R_2
Negato del confronto	CMN	CMN R_1, R_2	NZCV ← R_1 +R_2
Test equales	TEQ	TEQ R_1, R_2	NZCV ← $R_1 \oplus R_2$
Test	TST	TST R_1, R_2	NZCV ← $R_1 \wedge R_2$

Tabella variabili di condizione

Suffisso	Descrizione	Flag testato
EQ	Equal (==)	Z = 1
NE	Not Equal (!=)	Z = 0
CS / HS	Unsigned maggiore uguale (>=)	C = 1
CC / LO	Unisgned minore stretto (<)	C = 0
MI	Negativo (< 0)	N = 1
PL	Positivo o 0 (>= 0)	N = 0
VS	Overflow	V = 1
VC	No overflow	V = 0
HI	Unsigned maggiore stretto (>)	C = 1 & Z = 0
LS	Unsigned minore uguale (<=)	C = 0 & Z = 1
GE	Maggiore uguale (>=)	N = V
LT	Minore stretto (<)	N != V
GT	Maggiore stretto (>)	Z = 0 & N = V
LE	Minore uguale (<=)	Z = 1 & N != V
AL	Always	----